

Informe final de pruebas de banco

Proyecto Semáforos Sisga — IT Vial SAS

Elaborado por	Ing. Marco Perez Ramirez
Unidad Maestro	Maestro 1
Unidad Esclavo	Esclavo 1
Tipo de ensayo	Prueba de banco
Paquete revisado	Paquete_Banco_2026-09-28_1e56d83_SIN_BANCO
Commit de firmware	1e56d83
Fecha de la sesión	2026-09-28 / 2026-09-29
Documento de pruebas base	05_Funcional/Pruebas_Funcional_2026-09-28.html
Firmware cargado	Maestro, Esclavo y ESP32 de expansión, compilados desde 01_Firmware de este mismo pa
App móvil cargada	IOT_VIAL_Semaforos_2026-09-28_1e56d83_SIN_BANCO.apk

1. Introducción y alcance

Este informe documenta la sesión de pruebas de banco realizada sobre la actualización del proyecto Semáforos Sisga identificada como **Paquete_Banco_2026-09-28_1e56d83_SIN_BANCO**, distinta y posterior a la revisión trabajada anteriormente (V9.0, 2026-09-08, commit a868205). Al inicio de la sesión el cliente ya había cargado, desde el mismo árbol de código incluido en este paquete (carpeta 01_Firmware), el firmware nuevo de la tarjeta Maestro, el firmware nuevo de la tarjeta Esclavo, el firmware nuevo de los módulos ESP32 de expansión de ambas tarjetas, y la actualización correspondiente de la aplicación móvil (IOT_VIAL_Semaforos_2026-09-28_1e56d83_SIN_BANCO.apk).

El paquete incluye un documento de pruebas funcionales en formato HTML (05_Funcional/Pruebas_Funcional_2026-09-28.html) con nueve pruebas numeradas de la 0 a la 8, cada una con un procedimiento paso a paso, un resultado esperado, y un campo de respuesta (Funcionó / No funcionó / No se pudo probar) más un cuadro de texto libre para describir lo observado. Este informe sigue ese mismo documento como guía, prueba por prueba, y además incorpora la verificación de cada resultado contra el código fuente real del firmware cargado, de modo que las conclusiones no dependen únicamente de lo que muestra la aplicación en pantalla, sino de lo que el firmware efectivamente hace por dentro.

Por decisión explícita del cliente, la **Prueba 0 (versión del firmware)** no se realizó en esta sesión y queda excluida del alcance de este informe. Las Pruebas 1 a 8 se ejecutaron en su totalidad, con el detalle que se describe en la sección 4.

2. Metodología

Cada prueba se ejecutó siguiendo el procedimiento descrito en el documento funcional del paquete, con dos particularidades propias de un banco de pruebas de laboratorio (no una instalación de campo): en primer lugar, el banco no dispone todavía de la barrera mecánica (talanquera) física, por lo que la Prueba 3 se validó por lógica pura, leyendo directamente el campo PLUMA: de la trama de estado que publica el equipo (ARRIBA / ABAJO), lo cual es equivalente a medir con un multímetro la salida física del conector J15, ya que la lógica que decide subir o bajar la pluma es idéntica exista o no un motor conectado a esa salida. En segundo lugar, el banco solo cuenta con un conector de alimentación disponible para las cámaras, así que las cuatro cámaras Hikvision del proyecto se probaron una por una (una sola cámara alimentada a la vez), lo que además tuvo la ventaja de dar resultados sin ambigüedad para cada cámara individual, evitando el efecto de enmascaramiento del campo CAM: combinado que se describe en la Prueba 1.

En cada punto en que la respuesta de la aplicación no bastaba por sí sola para explicar lo observado, se consultó el código fuente del firmware realmente cargado en las tarjetas (incluido en este mismo paquete) para confirmar el mecanismo exacto detrás de cada comportamiento. Esa verificación aparece citada en el texto de cada prueba, con el archivo y, cuando corresponde, la línea de código o el mensaje literal exacto que el equipo devuelve.

3. Resumen de resultados

Prueba	Resultado
0. Versión del firmware	Omitida por decisión del cliente
1. Cámara sin cámara (puente J16)	Funcionó
2. Cámara real (Hikvision)	Funcionó
3. Talanquera con cámara	Funcionó (validada por lógica, sin talanquera física)
4. Prueba de focos	Funcionó
5. Ámbar pedido	Funcionó
6. Prueba de enlace	Funcionó
7. Degradado	Funcionó
8. Reloj	Funcionó (reconciliado — ver sección 4.8)

De las nueve pruebas del documento, ocho se completaron con resultado **Funcionó**. La Prueba 0 no se realizó por decisión del cliente. Durante la ejecución se identificaron y documentaron cuatro particularidades de diseño del firmware que conviene tener presentes para la operación e instalación del sistema (detalladas en la sección 5): el campo combinado de estado de cámaras, el mecanismo de veto de la barrera, el alcance limitado de la prueba de focos, y la arquitectura real del reloj del equipo. Ninguna de las cuatro impidió que la prueba correspondiente se cerrara como Funcionó, porque en los cuatro casos se trata de comportamiento intencional del firmware y no de un defecto.

4. Desarrollo detallado de las pruebas

4.0. Prueba 0 — Versión del firmware

OMITIDA

Esta prueba consiste en pedir a cada tarjeta, por el menú correspondiente, el comando "Consultar versión del firmware" y anotar literalmente el texto o código que contesta. Por decisión explícita del cliente, no se realizó en esta sesión y no forma parte de los resultados de este informe.

4.1. Prueba 1 — Cámara sin cámara (puente en J16)

FUNCIONÓ

El propósito de esta prueba es comprobar, sin necesidad de tener las cámaras reales conectadas todavía, que cada una de las dos entradas de cámara de cada tarjeta (los pines 9-10 y 11-12 del conector J16) está siendo correctamente vigilada por el firmware. Para eso se simula el cierre de contacto que produce una cámara real puenteando brevemente esos pines con un cable, y se observa si el campo de estado de cámaras de la aplicación pasa de signo de interrogación a la palabra "OK".

En el **Maestro** se hizo primero el puente entre el pin 10 y el pin 9 (la primera entrada de cámara, identificada en el firmware como cámara C): el campo de estado pasó de "?" a "OK" correctamente. A continuación se hizo el puente entre el pin 12 y el pin 11 (la segunda entrada, cámara D), y esta vez no se observó ningún cambio visible en la aplicación. En el **Esclavo**, con el fin de verificar la segunda entrada de forma aislada y sin depender del resultado de la primera, se invirtió el orden: se probó primero el puente 12-11 (cámara D), que sí hizo pasar el campo de "?" a "OK", y después el puente 10-9 (cámara C), que de nuevo no mostró cambio visible.

Esta aparente falta de reacción de la segunda cámara en cada tarjeta no es un defecto: se explica por cómo está construido el campo de estado en el firmware. La función correspondiente en el código fuente (`botones.cpp`, función `camara_estado()`) publica *la peor de las dos cámaras, pero únicamente entre aquellas que ya han dado al menos una señal*. Mientras una entrada nunca ha recibido una transición de nivel, queda excluida de esa comparación; solo se muestra "?" cuando ninguna de las dos ha hablado todavía. En cuanto una de las dos entradas informa "OK" por primera vez, el campo combinado ya puede decir "OK", aunque la otra entrada siga sin haber sido probada — y ese estado, una vez alcanzado, no vuelve a bajar. Por diseño, además, el firmware no genera ningún evento ni mensaje adicional cuando una segunda cámara pasa de "nunca probada" a "OK": solo se registran eventos ante fallas (cámara ciega o cámara pegada), nunca ante una transición exitosa a OK.

Precisamente por eso, en el Esclavo se optó deliberadamente por probar primero la segunda entrada (cámara D) sin que la primera (cámara C) hubiera hablado antes: de esa forma, el cambio observado de "?" a "OK" solo puede atribuirse a la cámara D, confirmando sin ambigüedad que el pin 12 del Esclavo también está siendo vigilado por el firmware y no es una entrada muerta que simplemente queda enmascarada por la otra. Con este cruce de pruebas (C primero en el Maestro, D primero en el Esclavo) quedaron confirmadas, de forma independiente y sin ambigüedad, las cuatro entradas de cámara del sistema. En ningún caso ninguna entrada quedó marcada como "PEGADA" (contacto trabado).

Queda documentada, para tenerla presente en la operación del sistema, una limitación de diseño del campo de estado: al ser un único campo por tarjeta que solo informa el peor de los dos casos, un operador que revise la aplicación no puede saber, únicamente a partir de ese campo, cuál de las dos cámaras de un mismo poste está fallando cuando la otra funciona correctamente; para diagnosticarlo haría falta revisar

físicamente cada cámara por separado.

4.2. Prueba 2 — Cámara real (Hikvision DS-2CD2683G2-IZS)

FUNCIONÓ

Confirmadas las cuatro entradas eléctricas en la Prueba 1, se cableó cada cámara real a su entrada correspondiente de J16 usando el borne de salida de alarma (contacto seco, terminales 1A/1B) de la cámara Hikvision. Debido a que el banco solo dispone de un único conector de alimentación para las cámaras, las cuatro se probaron una por una: se alimentaba una sola cámara a la vez, se pasaba caminando delante de ella con el equipo en modo Inteligente, se anotaba el resultado, y se pasaba el conector de alimentación a la siguiente.

Se probaron así las cuatro combinaciones del proyecto: Maestro cámara 1 (IP 192.168.1.61, pines 9-10), Maestro cámara 2 (IP 192.168.1.63, pines 11-12), Esclavo cámara 1 (IP 192.168.1.62, pines 9-10) y Esclavo cámara 2 (IP 192.168.1.64, pines 11-12). En las cuatro, al aparecer el vehículo frente a la cámara — visible en la interfaz de la cámara como un recuadro de color rojo, que corresponde a la clasificación "Vehículo" de su motor de análisis — la aplicación mostró el campo de cámaras en "OK", replicando exactamente el comportamiento ya validado con el puente simulado de la Prueba 1. Las cuatro cámaras cumplen así el criterio de esta prueba.

4.3. Prueba 3 — Talanquera con cámara

FUNCIONÓ

El banco de pruebas no cuenta todavía con la barrera mecánica (talanquera) instalada, por lo que esta prueba se adaptó para validar la lógica de control sin depender del actuador físico. El firmware publica en su trama de estado un campo dedicado, `PLUMA:`, con los valores `ARRIBA` o `ABAJO` — confirmado en el código fuente (`bluetooth.cpp`) que ese valor refleja exactamente lo último que la lógica de control escribió en la salida del motor de la talanquera, exista o no un motor real conectado a esa salida. Verificar ese campo es, por tanto, equivalente a medir con un multímetro la tensión presente en el conector de salida J15.

El primer intento de esta prueba, con el semáforo ya asentado en rojo desde hacía un rato, no produjo ningún cambio al poner un objeto delante de la cámara: la pluma permaneció indicando "ABAJO" sin importar la detección. Antes de considerar esto una falla se revisó el código fuente de control de la pluma (`semaforo.cpp`) y se encontró la explicación exacta, que resultó ser un comportamiento deliberado del diseño y no un defecto: **una cámara puede impedir que la pluma termine de bajar, pero nunca puede volver a levantar una pluma que ya se asentó en la posición ABAJO**. El propio comentario del código lo expresa así: "una cámara no puede levantar una pluma, igual que no puede encender un verde" — la única fuente que puede subir la barrera es la luz verde del semáforo; la cámara solo tiene capacidad de veto sobre una bajada que todavía está en curso.

El firmware define además una ventana de cortesía fija de tres segundos (`PLUMA_RETARDO_BAJADA_MS = 3000` milisegundos) inmediatamente después de que la luz cambia a rojo: durante esos tres segundos la pluma permanece arriba sin que el firmware consulte a la cámara, dando tiempo a que quien haya entrado legalmente con el verde termine de cruzar. Solo una vez agotada esa ventana, el firmware pregunta a la cámara del poste (`camara_presenciaJ16()`, que combina las dos entradas del poste con un OR lógico — basta que una de las dos vea algo para vetar la bajada) si debe seguir vetando o si ya puede bajar. Repitiendo la prueba con el objeto colocado justo en el instante en que la luz cambió a rojo, dentro de esa

ventana de tres segundos, el resultado fue el esperado: la pluma se mantuvo en "ARRIBA" mientras la cámara seguía detectando, y pasó a "ABAJO" poco después de retirar el objeto.

Repitiendo el mismo procedimiento, con el mismo cuidado en el instante de colocar el objeto, en las otras tres combinaciones de tarjeta y cámara (Maestro cámara 2, Esclavo cámara 1 y Esclavo cámara 2), las cuatro mostraron idéntico comportamiento: veto correcto mientras hay detección, liberación correcta poco después de que la cámara deja de ver algo. La prueba se considera **Funcionó** en las cuatro combinaciones, con la salvedad de que queda pendiente repetirla con el motor y el brazo real de la talanquera conectados, en cuanto estén disponibles en el banco, para confirmar además el movimiento mecánico y no solo la lógica de control.

4.4. Prueba 4 — Prueba de focos

FUNCIONÓ

Con el Maestro puesto fuera de servicio (modo Menú) se lanzó la prueba de focos desde el menú del equipo. La secuencia de luces observada fue la esperada: rojo, ámbar y verde en ese orden, y al terminar la prueba la luz regresó correctamente a la que correspondía según el estado del ciclo. Además, y como pide expresamente el procedimiento, se intentó lanzar la misma prueba con el equipo en modo Automático (en servicio): el equipo la rechazó, tal como ya se había observado de forma informal en una etapa anterior de la sesión. En esta ocasión se confirmó, contra el código fuente del firmware, el mensaje literal exacto que produce ese rechazo:

```
$ERR,CMD:TEST_LEDS,DESC:EN_SERVICIO_PASE_A_MENU
```

Al intentar repetir el mismo procedimiento en la tarjeta Esclavo se encontró que la prueba de focos no está disponible en absoluto para esa tarjeta: cualquier intento se rechaza siempre, con el mensaje literal:

```
$ERR,CMD:TEST_LEDS,DESC:NO_EN_SERVICIO_USE_EL_MAESTRO
```

Revisando el código de navegación de pantalla del Esclavo se confirmó además que, en este firmware, el Esclavo ya no tiene ninguna pantalla o menú físico propio: la interfaz de cinco pantallas que existía antes fue retirada por completo, y las funciones correspondientes quedaron vacías a propósito. Esto significa que, tal como está construido hoy el sistema, no existe absolutamente ninguna vía — ni por la aplicación ni por una pantalla física — para probar individualmente las lámparas del Esclavo, una por una y fuera de servicio; la única forma de verificarlas es observarlas encenderse durante la operación normal del ciclo. Se documenta este vacío funcional como una particularidad del diseño actual, sin que constituya un defecto de esta sesión de pruebas: la prueba de focos, tal como existe, fue diseñada para ejercitar únicamente las tres lámparas del Maestro.

Como dato adicional confirmado en el código, la prueba de focos del Maestro exige dos condiciones de seguridad antes de arrancar: que el Esclavo esté confirmado en rojo (de lo contrario responde ESPERANDO_ROJO_DEL_ESCLAVO), y que el equipo no esté en el estado de ámbar intermitente por pérdida de enlace de radio (de lo contrario responde SIN_ENLACE_AMBAR_NO_SE_PRUEBA). Ambas condiciones se cumplieron durante la prueba realizada.

4.5. Prueba 5 — Ámbar pedido

FUNCIONÓ

Se puso el equipo en modo Ámbar desde la aplicación y se verificó que ésta deja de marcarlo como un fallo de comunicación, mostrando en cambio el ámbar como un estado solicitado deliberadamente. El resultado

fue el esperado.

4.6. Prueba 6 — Prueba de enlace

FUNCIONÓ

Se pulsó "Prueba de Alcance" en la aplicación con los dos postes del banco separados aproximadamente un metro. La pantalla mostró en grande el porcentaje de calidad de enlace, con un valor del 90%. Se deja constancia de que, a una distancia tan corta como la del banco, este porcentaje debe entenderse únicamente como una comprobación de referencia de que el mecanismo de medición funciona: a esa cercanía el radio puede incluso saturarse, por lo que el valor no es representativo del porcentaje de enlace que se obtendrá en campo con la separación real entre los dos postes del cruce.

4.7. Prueba 7 — Degradado

FUNCIONÓ

El procedimiento seguido fue el siguiente: primero se sincronizó la hora en los dos postes, empezando por el Maestro y siguiendo con el Esclavo; después se activó el Modo Degradado, también primero en el Maestro (pulsando la opción correspondiente) y luego en el Esclavo (pulsando la misma opción en su propia interfaz). Al activarse, el equipo mostró el letrero "Degradado arrancado, todo rojo primero, compruebe el otro poste", y las dos tarjetas quedaron efectivamente en rojo por la orden de entrada al modo. A partir de ahí, el ciclo comenzó a alternar de forma autónoma entre las dos puntas: Maestro en verde con Esclavo en rojo, y después Maestro en rojo con Esclavo en verde, y así sucesivamente, con una duración aproximada de tres minutos de verde para cada lado y veinticinco segundos de todo-rojo (el margen de despeje) entre cada cambio de fase.

Ese margen de veinticinco segundos corresponde al parámetro de despeje configurado en el ciclo del Modo Degradado (`ciclo_degradado.h`), y coincide numéricamente — aunque sin relación alguna entre ambos mecanismos — con el tiempo de silencio de radio que hace que el Esclavo caiga a ámbar intermitente cuando pierde contacto con el Maestro, descrito en la sección 4.8. Se trata de dos parámetros de diseño independientes que simplemente comparten el mismo valor numérico en la configuración actual.

El documento de pruebas pide, además, exportar justo después de esta prueba la cinta de tramas y el diario de órdenes de la aplicación de las dos tarjetas. El cliente cuenta con esos archivos capturados para todas las pruebas de esta sesión y los comparte por un canal aparte; no forman parte de este informe ni fueron analizados durante esta sesión de trabajo.

4.8. Prueba 8 — Reloj

FUNCIONÓ

Esta prueba merece una explicación más extensa porque, durante la sesión, surgió un resultado que a primera vista parecía un defecto y que un análisis más profundo del firmware permitió aclarar por completo.

Al pedir, desde la aplicación, la opción "Reiniciar el reloj del equipo" en el Maestro, éste respondió con el mensaje "el reloj sigue parado: los bits están en registro de eventos". Contrastando ese mensaje contra el código fuente del firmware realmente cargado (`bluetooth.cpp`) se confirmó que la respuesta real y completa del equipo es la siguiente, un mensaje de error y no de confirmación:

```
$ERR,CMD:REINICIAR_RELOJ,DESC:SIGUE_PARADO_VEA_CONSULTA_RELOJ
```

Ese error significa, en términos de hardware, que el oscilador de cristal del reloj de tiempo real interno del microcontrolador STM32 del Maestro (identificado en los planos como Y2) no llega a arrancar tras el intento

de reinicio. Antes de registrar esto como un defecto pendiente de diagnosticar, se revisó con detenimiento el código fuente del módulo de reloj (`reloj.cpp`), y ese mismo archivo documenta explícitamente, en sus comentarios de cabecera, que **este microcontrolador no tiene garantizado un cristal de 32.768 kHz funcional** ("Y2 muerto") y que, precisamente por eso, **la autoridad real de la hora del sistema no es ese cristal interno, sino el módulo ESP32 con su reloj DS3231 que lleva cada poste**. El microcontrolador STM32 mantiene, en su lugar, una base de tiempo calculada por software a partir de la hora que le siembra el ESP32, y esa base de software es la que en la práctica gobierna toda la lógica que depende de la hora, incluida la fase del Modo Degradado.

Dicho de otro modo: el cristal Y2 del Maestro sin arrancar no es un defecto nuevo introducido por esta actualización de firmware, sino una condición de hardware ya conocida por quienes diseñaron el sistema, para la que existe una solución de diseño deliberada y ya implementada. Lo que de verdad hay que verificar no es si ese cristal en particular arranca, sino si el mecanismo de respaldo (ESP32 más DS3231) efectivamente mantiene la hora correcta del equipo, incluso a través de un corte real de energía.

Para comprobar precisamente eso, se realizó la siguiente prueba, que cubre el procedimiento pedido por el documento funcional: se sincronizó la hora en el Maestro y en el Esclavo a partir de la hora del teléfono; se desconectó por completo la alimentación de las dos tarjetas; se esperó cinco minutos con el equipo apagado; se volvió a conectar la alimentación; y, en las dos tarjetas, se pidió la opción "Consultar reloj". El equipo no imprime un valor de desfase calculado como tal — la verificación se hace comparando visualmente la hora que muestra el teléfono (la fuente que se usó para sincronizar) contra la hora que el equipo informa tener. En esta prueba, la hora del Maestro y la del Esclavo coincidieron con la del teléfono, con una diferencia visual de aproximadamente un segundo en ambos casos — una precisión perfectamente razonable después de cinco minutos sin energía y coherente con la que se espera de un módulo DS3231.

Con esa evidencia, la conclusión de esta prueba es que el reloj del equipo **Funcionó**: el mecanismo real de conservación de la hora, que es el que importa para la seguridad de la operación del cruce, demostró mantener la hora correcta a través de un corte de energía real en las dos tarjetas. El mensaje de error del cristal interno Y2 del Maestro queda documentado como una característica conocida del hardware, sin impacto sobre la hora efectiva del sistema.

5. Particularidades de diseño documentadas en esta sesión

A lo largo de las pruebas se identificaron cuatro comportamientos del firmware que, sin ser fallas, conviene tener presentes tanto para la operación del sistema en campo como para la redacción de manuales de uso, porque de otro modo un instalador o un operador podría interpretarlos erróneamente como averías.

5.1 — *El campo de estado de cámaras es único por tarjeta*

El campo CAM: que muestra la aplicación combina el estado de las dos cámaras de un mismo poste en un solo valor, mostrando siempre el peor de los dos casos entre las cámaras que ya han dado alguna señal. Esto significa que, si una de las dos cámaras de un poste falla mientras la otra sigue funcionando, la aplicación seguirá mostrando "OK" y no dará ninguna pista de cuál de las dos está fallando. La verificación de cuál cámara específica falló requiere inspección física en el poste.

5.2 — *La cámara solo puede vetar, nunca levantar la barrera*

Una vez que la barrera ha terminado de bajar, ninguna detección posterior de una cámara puede volver a subirla: la única acción que puede subir la barrera es la luz verde del semáforo. Las cámaras únicamente pueden retener la barrera arriba mientras la bajada todavía está en curso, dentro de una ventana de cortesía de tres segundos tras el cambio a rojo. Este comportamiento es intencional y está pensado para que la barrera no atropelle a un vehículo o una persona que quedó bajo ella durante los primeros segundos tras el cambio de luz, pero no convierte a la cámara en un sensor de presencia general para cualquier momento posterior.

5.3 — *La prueba de focos solo cubre las lámparas del Maestro*

La función de diagnóstico que enciende las tres luces en secuencia fuera de servicio existe únicamente en el Maestro. El Esclavo la rechaza siempre y, además, ya no cuenta con pantalla física propia en este firmware. No existe hoy ninguna manera de verificar individualmente las lámparas del Esclavo fuera del funcionamiento normal del ciclo.

5.4 — *La hora real del sistema depende del ESP32/DS3231, no del cristal del STM32*

El cristal de reloj interno del microcontrolador de cada tarjeta puede no arrancar — condición conocida del hardware y prevista en el diseño — sin que esto comprometa la hora del sistema, porque esa responsabilidad recae en el módulo ESP32 con reloj DS3231 de cada poste. Cualquier evaluación futura del reloj del sistema debería centrarse en verificar ese mecanismo de respaldo, y no el cristal interno del STM32 por separado.

6. Conclusiones

De las nueve pruebas contempladas en el documento funcional del paquete Paquete_Banco_2026-09-28 (commit 1e56d83), ocho se ejecutaron por completo con resultado satisfactorio: detección de cámaras simulada y real en las cuatro combinaciones del proyecto, comportamiento correcto del veto de la barrera validado por lógica, secuencia de prueba de focos del Maestro en el orden correcto, modo Ámbar sin marcarse como fallo de comunicación, enlace de radio funcionando y midiendo su calidad, Modo Degradado alternando correctamente entre los dos postes, y conservación correcta de la hora del sistema a través de un corte real de energía. La única prueba no realizada, la de versión de firmware, quedó excluida por decisión explícita del cliente.

No se identificó, en esta sesión, ningún defecto de firmware que impida avanzar con la siguiente etapa del proyecto. Las cuatro particularidades de diseño descritas en la sección 5 no son fallas, pero se recomienda incorporarlas explícitamente en la documentación de instalación y en la capacitación del personal de campo, para evitar diagnósticos erróneos el día que se observen en una instalación real.

7. Pendientes para próximas sesiones

- Repetir la Prueba 3 (talanquera) con el motor y el brazo real conectados, para confirmar el movimiento mecánico además de la lógica ya validada.
- Cableado físico definitivo de J16 para Maestro cámara 2, Esclavo cámara 1 y Esclavo cámara 2 (ya validadas eléctricamente en banco en esta y en sesiones anteriores; falta el montaje final fuera del banco de pruebas).
- Incorporar en el manual de instalación y en la capacitación de campo las cuatro particularidades de diseño descritas en la sección 5 de este informe.

Informe elaborado a partir de las pruebas ejecutadas en banco por el Ing. Marco Perez Ramirez sobre las unidades Maestro 1 y Esclavo 1, y verificado contra el código fuente del firmware incluido en el paquete Paquete_Banco_2026-09-28_1e56d83_SIN_BANCO.